

AHK Academy - API Reference

Complete reference for all API endpoints. Base URL: `http://localhost:3000/api`
Auth: All protected routes use NextAuth.js session cookies. Roles: ADMIN, TEACHER, STUDENT.

Table of Contents

- 1. [Auth](#)
- 2. [Students](#)
- 3. [Teachers](#)
- 4. [Classes & Enrollments](#)
- 5. [Lessons & Assignments](#)
- 6. [Grammar System](#)
- 7. [Speaking Sessions](#)
- 8. [Topics](#)
- 9. [AI Analysis & Homework](#)
- 10. [Payments](#)
- 11. [WhatsApp](#)
- 12. [Blog](#)
- 13. [Notifications](#)
- 14. [Onboarding](#)
- 15. [Teacher Resources](#)
- 16. [Cron Jobs](#)
- 17. [Webhooks](#)
- 18. [AI Chat](#)

Auth

GET | POST `/api/auth/[...nextauth]`

NextAuth.js handler. Manages sign-in, sign-out, session, and CSRF.

Detail	Value
Auth	None (public)
Description	NextAuth catch-all route for OAuth/credentials sign-in flows

Key sign-in credentials (dev seed):

- Admin: `ahsan@ahkacademy.com / admin123`
- Teacher: `brishna@ahkacademy.com / teacher123`
- Student: `student@ahkacademy.com / student123`

Students

GET `/api/students`

Detail	Value
Auth	None (admin-facing)
Description	List all students ordered by creation date
Response	Array of User objects with <code>_count</code> : { <code>enrollments</code> , <code>payments</code> }

POST `/api/students`

Detail	Value
Auth	None (admin-facing)
Description	Create a new student account

Request Body:

```
{
  "name": "string (required)",
  "email": "string (required)",
  "phone": "string (optional)",
  "password": "string (required)"
}
```

Response: 201 - Created User object

DELETE `/api/students?id=xxx`

Detail	Value
Auth	None (admin-facing)
Description	Delete a student by ID
Query Params	<code>id</code> (required)
Response	{ <code>success</code> : true }

GET `/api/students/[id]`

Detail	Value
Auth	None (admin-facing)
Description	Get full student profile with enrollments, payments, attendances, submissions

Response:

```
{
  "id": "string",
  "name": "string",
  "email": "string",
  "enrollments": [{ "class": { "course": {}, "teacher": {} } }],
  "payments": [],
  "attendances": [{ "lesson": {} }],
  "submissions": [{ "assignment": {} }]
}
```

Teachers

GET /api/teachers

Detail	Value
Auth	None (admin-facing)
Description	List all teachers with class count
Response	Array of User objects with _count: { teacherClasses }

POST /api/teachers

Detail	Value
Auth	None (admin-facing)
Description	Create a new teacher account

Request Body:

```
{
  "name": "string (required)",
  "email": "string (required)",
  "phone": "string (optional)",
  "password": "string (required)"
}
```

Response: 201 - Created User object

GET /api/teacher/classes

Detail	Value
Auth	Authenticated (session-based, returns own classes)
Description	Get classes belonging to the logged-in teacher
Response	Array of Class objects with course info and counts

GET /api/teacher/students?classId=xxx

Detail	Value
Auth	None (teacher-facing)
Description	Get active students enrolled in a specific class
Query Params	classId (required)
Response	Array of { id, name, email, phone }

Classes & Enrollments

GET /api/classes

Detail	Value
Auth	None (admin-facing)
Description	List all classes with course, teacher, and enrollment count

POST /api/classes

Detail	Value
Auth	None (admin-facing)
Description	Create a new class

Request Body:

```
{
  "name": "string (required)",
  "courseId": "string (required)",
  "teacherId": "string (required)",
  "schedule": "string (optional)",
  "maxStudents": "number (default 30)",
  "meetingLink": "string (optional)"
}
```

GET /api/courses

Detail	Value
Auth	None (admin-facing)
Description	List all courses with class count

POST /api/courses

Detail	Value
Auth	None (admin-facing)
Description	Create a new course

Request Body:

```
{
  "title": "string (required)",
  "description": "string (optional)",
  "level": "string (default 'Beginner')",
  "price": "number (default 0)"
}
```

POST /api/enrollments

Detail	Value
Auth	None (admin-facing)
Description	Enroll a student in a class (upserts to ACTIVE)

Request Body:

```
{
  "studentId": "string (required)",
  "classId": "string (required)"
}
```

DELETE /api/enrollments?studentId=xxx&classId=xxx

Detail	Value
Auth	None (admin-facing)
Description	Remove a student from a class
Query Params	studentId, classId (both required)

GET /api/student/classes

Detail	Value
Auth	STUDENT (session)
Description	Get the logged-in student's enrollments with class/course/teacher details

Lessons & Assignments

GET /api/lessons?classId=xxx

Detail	Value
Auth	None
Description	Get all lessons for a class, ordered by lesson order
Query Params	classId (required)

POST /api/lessons

Detail	Value
Auth	None (teacher/admin-facing)
Description	Create a new lesson for a class

Request Body:

```
{
  "classId": "string (required)",
  "title": "string (required)",
  "content": "string (optional)",
  "topics": "string (optional)",
  "videoUrl": "string (optional)",
  "date": "ISO date string (optional)"
}
```

GET /api/student/lessons

Detail	Value
Auth	STUDENT (session)
Description	Get all lessons for the student's enrolled classes with attendance status
Response	Array of { id, title, content, topics, order, date, attended: boolean }

GET /api/assignments

Detail	Value
Auth	None
Description	Get assignments. Filtered by <code>classId</code> or <code>studentId</code> query param
Query Params	<code>classId</code> (optional), <code>studentId</code> (optional)

POST /api/assignments

Detail	Value
Auth	None (teacher/admin-facing)
Description	Create a new assignment

Request Body:

```
{
  "classId": "string (required)",
  "title": "string (required)",
  "description": "string (optional)",
  "dueDate": "ISO date string (optional)"
}
```

GET /api/submissions

Detail	Value
Auth	None
Description	Get submissions filtered by <code>assignmentId</code> or <code>studentId</code>
Query Params	<code>assignmentId</code> (optional), <code>studentId</code> (optional)

POST /api/submissions

Detail	Value
Auth	None (student-facing)
Description	Submit (or re-submit) work for an assignment

Request Body:

```
{
  "assignmentId": "string (required)",
  "studentId": "string (required)",
  "content": "string (optional)"
}
```

PATCH /api/submissions

Detail	Value
Auth	None (teacher-facing)
Description	Grade a submission

Request Body:

```
{
  "id": "string (required - submission ID)",
  "grade": "number (required)",
  "feedback": "string (optional)"
}
```

GET /api/attendance

Detail	Value
Auth	None
Description	Get attendance records filtered by <code>lessonId</code> or <code>studentId</code>
Query Params	<code>lessonId</code> (optional), <code>studentId</code> (optional)

POST /api/attendance

Detail	Value
Auth	None (teacher-facing)
Description	Mark attendance for multiple students in a lesson

Request Body:

```
{
  "records": [
    { "studentId": "string", "lessonId": "string", "status": "PRESENT|ABSENT|LATE" }
  ],
  "markedById": "string (teacher ID)"
}
```

GET /api/student/payments

Detail	Value
Auth	STUDENT (session)

Description Get the logged-in student's payment history

Grammar System

GET /api/grammar/categories

Detail	Value
Auth	Optional (students see progress stats)
Description	List all grammar categories with topic counts and student progress

Response:

```
{
  "categories": [{
    "id": "string",
    "name": "string",
    "slug": "string",
    "description": "string",
    "icon": "string",
    "order": "number",
    "topicCount": "number",
    "completedCount": "number",
    "masteredCount": "number"
  }]
}
```

GET /api/grammar/topics?categorySlug=xxx

Detail	Value
Auth	Optional (students see progress)
Description	List grammar topics for a category with progress tracking
Query Params	categorySlug (required)

Response:

```
{
  "topics": [{
    "id": "string",
    "title": "string",
    "slug": "string",
    "level": "string",
    "exerciseCount": "number",
    "progress": { "status": "not_started|in_progress|completed|mastered", "exerciseScore": 0, "quizScore": 0, "lessonRead": false }
  }]
}
```

GET /api/grammar/topics/[id]

Detail	Value
Auth	Optional (students see own progress/attempts)
Description	Get full topic detail with lesson content, exercises, and student progress

Response: Topic object with category, lesson (content, examples, tips), exercises[], and progress

POST /api/grammar/exercises/check

Detail	Value
Auth	STUDENT (session)
Description	Submit an answer to a grammar exercise and get feedback

Request Body:

```
{
  "exerciseId": "string (required)",
  "answer": "string (required)"
}
```

Response:

```
{
  "isCorrect": "boolean",
  "correctAnswer": "string",
  "explanation": "string"
}
```

GET /api/grammar/quiz/[topicId]

Detail	Value
Auth	Authenticated (session)
Description	Generate a randomized quiz (up to 10 questions) for a topic

Response:

```
{
  "topicId": "string",
  "topicTitle": "string",
  "totalQuestions": "number",
  "exercises": [{ "id", "type", "difficulty", "question", "options", "order" }]
}
```

POST /api/grammar/quiz/[topicId]

Detail	Value
--------	-------

Auth STUDENT (session)

Description Submit quiz answers and get score. Updates progress and marks assignments complete if score >= 70%

Request Body:

```
{
  "answers": [
    { "exerciseId": "string", "answer": "string" }
  ]
}
```

Response:

```
{
  "score": "number",
  "total": "number",
  "percentage": "number",
  "results": [{ "exerciseId", "isCorrect", "correctAnswer", "explanation" }]
}
```

GET /api/grammar/assignments

Detail	Value
--------	-------

Auth Authenticated (role-based)

Description Get grammar assignments. Students see own; teachers see theirs; admins see all

Query Params studentId (optional, for teacher/admin filtering)

POST /api/grammar/assignments

Detail	Value
--------	-------

Auth TEACHER or ADMIN

Description Assign a grammar topic to a student

Request Body:

```
{
  "studentId": "string (required)",
  "topicId": "string (required)",
  "dueDate": "ISO date string (optional)"
}
```

GET /api/grammar/progress

Detail	Value
--------	-------

Auth STUDENT (own) or TEACHER/ADMIN (with ?studentId=xxx)

Description Get overall grammar progress stats with category breakdown

Response:

```
{
  "totalTopics": "number",
  "completedTopics": "number",
  "masteredTopics": "number",
  "totalExerciseAttempts": "number",
  "averageScore": "number",
  "categoryBreakdown": [{ "id", "name", "slug", "totalTopics", "completedTopics", "masteredTopics" }]
}
```

PUT /api/grammar/progress

Detail	Value
--------	-------

Auth STUDENT

Description Mark a grammar lesson as read

Request Body:

```
{
  "topicId": "string (required)"
}
```

GET /api/grammar-lessons

Detail	Value
--------	-------

Auth TEACHER or ADMIN

Description Get grammar lesson logs. Teachers see own; admins see all (or filter with ?teacherId=xxx)
Query Params teacherId (optional, admin only)

POST /api/grammar-lessons

Detail	Value
Auth	TEACHER or ADMIN
Description	Log a grammar lesson that was taught

Request Body:

```
{
  "title": "string (required)",
  "grammarFocus": "string (required)",
  "description": "string (optional)",
  "taughtAt": "ISO date string (required)",
  "studentsTargeted": "string[] (optional)"
}
```

Speaking Sessions

GET /api/speaking-sessions

Detail	Value
Auth	Authenticated (role-filtered)
Description	List speaking sessions. ADMIN sees all; TEACHER sees own; STUDENT sees enrolled
Query Params	status (optional)
Response	Array of sessions with participants, transcript, creator

POST /api/speaking-sessions

Detail	Value
Auth	TEACHER or ADMIN
Description	Create a new speaking session

Request Body:

```
{
  "topic": "string (required)",
  "topicDetails": "string (optional)",
  "date": "ISO date string (required)",
  "duration": "number (default 90 minutes)",
  "meetingLink": "string (optional)",
  "maxStudents": "number (default 5)",
  "sourceType": "string (default 'system')",
  "sourceLessonId": "string (optional)",
  "topicId": "string (optional)",
  "participantIds": "string[] (optional)"
}
```

GET /api/speaking-sessions/[id]

Detail	Value
Auth	Authenticated
Description	Get full session detail with participants, transcript, analyses, creator, source lesson

PUT /api/speaking-sessions/[id]

Detail	Value
Auth	ADMIN or session creator
Description	Update session details

Request Body (all optional):

```
{
  "topic": "string",
  "topicDetails": "string",
  "date": "ISO date string",
  "duration": "number",
  "status": "scheduled|in_progress|completed|cancelled",
  "meetingLink": "string",
  "maxStudents": "number",
  "topicId": "string"
}
```

DELETE /api/speaking-sessions/[id]

Detail	Value
Auth	ADMIN or session creator
Description	Delete a speaking session

POST /api/speaking-sessions/[id]/participants

Detail	Value
Auth	TEACHER or ADMIN
Description	Add students to a session (respects maxStudents limit)

Request Body:

```
{
  "studentIds": ["string (required, array of student IDs)"]
}
```

DELETE /api/speaking-sessions/[id]/participants

Detail	Value
Auth	TEACHER or ADMIN
Description	Remove a student from a session

Request Body:

```
{
  "studentId": "string (required)"
}
```

PUT /api/speaking-sessions/[id]/attendance

Detail	Value
Auth	TEACHER or ADMIN
Description	Mark attendance for session participants

Request Body:

```
{
  "attendance": [
    { "studentId": "string", "attended": "boolean" }
  ]
}
```

POST /api/speaking-sessions/[id]/rate

Detail	Value
Auth	STUDENT (must be a participant)
Description	Rate a session (1-5 stars)

Request Body:

```
{
  "rating": "number (1-5, required)",
  "feedback": "string (optional)"
}
```

GET /api/speaking-sessions/[id]/transcript

Detail	Value
Auth	Authenticated
Description	Get the transcript for a session (with uploader info and analyses)

POST /api/speaking-sessions/[id]/transcript

Detail	Value
Auth	TEACHER or ADMIN
Description	Upload or update a session transcript

Request Body:

```
{
  "content": "string (required - full transcript text)",
  "fileUrl": "string (optional)"
}
```

POST /api/speaking-sessions/[id]/analyze

Detail	Value
Auth	TEACHER or ADMIN
Description	Trigger AI analysis of transcript for all attended participants. Creates analyses and teacher insights.
Prerequisites	Transcript must exist; at least one participant must have attended

Response:

```
{
  "analyses": [{ "studentId", "summary", "participationScore", "strengths", "areasToImprove", "homework" }],
  "insights": [{ "teacherId", "studentId", "type", "content", "priority" }],
  "message": "Analysis complete for N participants"
}
```


GET /api/student-speaking-profile

Detail	Value
Auth	STUDENT (session)
Description	Get (or auto-create) the student's speaking profile

PUT /api/student-speaking-profile

Detail	Value
Auth	STUDENT (session)
Description	Update speaking profile preferences

Request Body (all optional):

```
{
  "track": "speaking_only | full_student",
  "level": "beginner | intermediate | advanced",
  "preferredTopics": "string",
  "availability": "string"
}
```

GET /api/progress/[studentId]

Detail	Value
Auth	Authenticated (students can only view own)
Description	Comprehensive progress report for a student

Response:

```
{
  "student": { "id", "name", "email" },
  "totalSessionsAttended": "number",
  "averageParticipationScore": "number",
  "vocabularyGrowth": { "uniqueWordsCount": "number", "words": ["string"] },
  "grammarTimeline": [{ "sessionId", "topic", "date", "grammarNotes" }],
  "recentAnalyses": [{ "id", "summary", "participationScore", "strengths", "areasToImprove", "homework" }],
  "sessionHistory": [{ "sessionId", "topic", "date", "status", "attended", "rating", "feedback" }],
  "speakingProfile": {}
}
```

Topics

GET /api/topics

Detail	Value
Auth	Authenticated
Description	List speaking session topics. Filterable by category and level
Query Params	category (optional), level (optional)

POST /api/topics

Detail	Value
Auth	TEACHER or ADMIN
Description	Create a new discussion topic

Request Body:

```
{
  "title": "string (required)",
  "description": "string (optional)",
  "category": "string (optional)",
  "level": "string (default 'intermediate')",
  "preparationTips": "string (optional)",
  "isSystemGenerated": "boolean (default false)"
}
```

PUT /api/topics/[id]

Detail	Value
Auth	TEACHER or ADMIN
Description	Update a topic

DELETE /api/topics/[id]

Detail	Value
Auth	TEACHER or ADMIN
Description	Delete a topic

GET /api/teacher/topics

Detail	Value
Auth	TEACHER (session)

Description Get topics created by the logged-in teacher

POST /api/teacher/topics

Detail	Value
Auth	TEACHER (session)
Description	Create a topic (same fields as /api/topics POST)

AI Analysis & Homework

POST /api/ai/generate-homework

Detail	Value
Auth	None (admin/teacher-facing)
Description	Generate homework using Claude API (or template fallback if no API key)

Request Body:

```
{
  "lessonId": "string (optional)",
  "transcript": "string (optional)",
  "classId": "string (optional - if provided, creates Assignment record)"
}
```

Response:

```
{
  "homework": "string (formatted homework text)",
  "assignment": "Assignment object (if classId provided)"
}
```

POST /api/analysis/generate

Detail	Value
Auth	ADMIN or TEACHER
Description	Run full AI-powered analysis on a session transcript for all attended participants

Request Body:

```
{
  "sessionId": "string (required)"
}
```

Response:

```
{
  "sessionId": "string",
  "topic": "string",
  "analyzedStudents": "number",
  "analyses": [{
    "studentId": "string",
    "studentName": "string",
    "analysis": { "summary", "vocabularyUsed", "grammarNotes", "participationScore", "strengths", "areasToImprove", "homework" },
    "model": "string (AI model used)"
  }]
}
```

POST /api/homework/generate

Detail	Value
Auth	ADMIN or TEACHER
Description	Generate personalized homework based on a student's session analysis

Request Body:

```
{
  "sessionId": "string (required)",
  "studentId": "string (required)"
}
```

Response:

```
{
  "homework": {
    "id": "string",
    "title": "string",
    "sections": [{ "type": "grammar|vocabulary|writing|speaking|reading", "title", "instructions", "exercises": [] }],
    "estimatedMinutes": "number",
    "dueDate": "ISO date",
    "studentName": "string"
  },
  "model": "string"
}
```

GET /api/teacher-insights

Detail	Value
Auth	Authenticated (role-based)
Description	Get teacher insights. Teachers see insights for their students; students see insights about them
Query Params	studentId (optional, for teachers)

PUT /api/teacher-insights

Detail	Value
Auth	TEACHER or ADMIN (insight owner)
Description	Mark an insight as read

Request Body:

```
{
  "insightId": "string (required)"
}
```

POST /api/teacher/insights/mark-read

Detail	Value
Auth	TEACHER (session)
Description	Mark ALL unread insights as read for the logged-in teacher
Response	{ message: "All insights marked as read" }

Teacher Speaking Session Endpoints

POST /api/teacher/speaking

Detail	Value
Auth	TEACHER (session)
Description	Create a speaking session (teacher workflow)

Request Body:

```
{
  "topic": "string (required)",
  "topicDetails": "string (optional)",
  "date": "ISO date string (required)",
  "duration": "number (default 90)",
  "meetingLink": "string (optional)",
  "maxStudents": "number (default 5)",
  "sourceLessonId": "string (optional)",
  "studentIds": "string[] (optional)"
}
```

POST /api/teacher/speaking/[id]/transcript

Detail	Value
Auth	TEACHER (session)
Description	Upload transcript for a session

Request Body:

```
{
  "content": "string (required)"
}
```

POST /api/teacher/speaking/[id]/analyze

Detail	Value
Auth	TEACHER (session)
Description	Run analysis on session transcript. Creates per-student analyses and teacher insights.

POST /api/teacher/speaking/[id]/homework

Detail	Value
Auth	TEACHER (session)
Description	Generate personalized homework for a specific student from a session

Request Body:

```
{
  "studentId": "string (required)"
}
```

POST /api/teacher/speaking/[id]/attendance

Detail	Value
Auth	TEACHER (session)
Description	Save attendance for session participants

Request Body:

```
{
  "attendance": {
    "studentId1": true,
    "studentId2": false
  }
}
```

POST /api/teacher/speaking/[id]/assign-topic

Detail	Value
Auth	TEACHER (session owner only)
Description	Update the topic for a speaking session

Request Body:

```
{
  "topic": "string (required)"
}
```

Payments

GET /api/payments

Detail	Value
Auth	None (admin-facing)
Description	List all payments with student info

POST /api/payments

Detail	Value
Auth	None (admin-facing)
Description	Create a payment record

Request Body:

```
{
  "studentId": "string (required)",
  "amount": "number (required)",
  "paidAmount": "number (default 0)",
  "dueDate": "ISO date string (optional)",
  "method": "string (optional)",
  "description": "string (optional)"
}
```

PATCH /api/payments

Detail	Value
Auth	None (admin-facing)
Description	Record an additional payment against an existing record

Request Body:

```
{
  "id": "string (required - payment ID)",
  "paidAmount": "number (required - amount being paid now)",
  "method": "string (optional)"
}
```

GET /api/payments/plans

Detail	Value
Auth	Authenticated (role-based)
Description	List payment plans. ADMIN sees all; TEACHER sees assigned students'; STUDENT sees own

POST /api/payments/plans

Detail	Value
Auth	ADMIN only
Description	Create a payment plan for a student. Auto-generates reference code.

Request Body:

```
{
  "studentId": "string (required)",
  "planType": "string (required - e.g. 'speaking_only', 'full_track')",
  "amount": "number (required)",
  "currency": "string (default 'USD')",
  "billingCycleDay": "number (optional - day of month)",
  "notes": "string (optional)"
}
```

GET /api/payments/transactions

Detail	Value
Auth	Authenticated (ADMIN sees all, STUDENT sees own)
Description	List payment transactions
Query Params	studentId (optional, admin), status (optional: PENDING, CONFIRMED)

POST /api/payments/transactions

Detail	Value
Auth	ADMIN only
Description	Create a new payment transaction with auto-generated reference code

Request Body:

```
{
  "studentId": "string (required)",
  "amount": "number (required)",
  "method": "string (required - IBAN, PAPARA, STRIPE)",
  "description": "string (optional)",
  "sessionId": "string (optional)"
}
```

POST /api/payments/confirm

Detail	Value
Auth	ADMIN only
Description	Confirm a pending manual/IBAN payment. Sends email receipt to student.

Request Body:

```
{
  "transactionId": "string (required)"
}
```

WhatsApp

POST /api/webhooks/whatsapp

Detail	Value
Auth	None (webhook from 360dialog/Meta)
Description	Receives inbound WhatsApp messages. For unknown users (leads), triggers AI chatbot response. For known users, logs message.

GET /api/webhooks/whatsapp

Detail	Value
Auth	None (Meta webhook verification)
Description	Webhook verification endpoint. Validates hub.verify_token
Query Params	hub.mode, hub.verify_token, hub.challenge

POST /api/whatsapp/send

Detail	Value
Auth	ADMIN or TEACHER
Description	Send a WhatsApp message or template to a phone number

Request Body:

```
{
  "phone": "string (required)",
  "message": "string (required if no templateName)",
  "templateName": "string (optional - use instead of message)",
  "variables": "string[] (optional - template variables)"
}
```

GET /api/whatsapp/conversations

Detail	Value
Auth	ADMIN only
Description	List all WhatsApp conversations with last message
Query Params	type (optional: LEAD, STUDENT), search (optional: phone/name/email)

GET /api/whatsapp/conversations/[id]

Detail	Value
Auth	ADMIN only
Description	Get full conversation detail with all messages

Blog

GET /api/blog

Detail	Value
Auth	Public (sees published only); ADMIN (sees all with status filter)
Description	List blog posts
Query Params	language (optional: en, tr), status (optional, admin: DRAFT, PUBLISHED)

POST /api/blog

Detail	Value
Auth	ADMIN only
Description	Create a new blog post

Request Body:

```
{
  "title": "string (required)",
  "content": "string (required - markdown)",
  "excerpt": "string (optional - auto-generated from content)",
  "language": "string (default 'en')",
  "status": "string (default 'DRAFT' - or 'PUBLISHED')",
  "keywords": "string (optional)",
  "metaTitle": "string (optional)",
  "metaDescription": "string (optional)",
  "featuredImage": "string (optional - URL)"
}
```

POST /api/blog/generate

Detail	Value
Auth	ADMIN only
Description	AI-generate a full blog post (SEO-optimized). Created as DRAFT.

Request Body:

```
{
  "topic": "string (required)",
  "language": "string (default 'en')",
  "targetKeyword": "string (optional)"
}
```

Response:

```
{
  "post": { "id", "title", "slug", "content", "excerpt", "metaTitle", "metaDescription", "language", "status" },
  "model": "string (AI model used)"
}
```

Notifications

GET /api/notifications

Detail	Value
Auth	Authenticated (returns own notifications)
Description	Get notifications for the logged-in user (latest 50)
Query Params	unread=true (optional - filter to unread only)

PUT /api/notifications

Detail	Value
Auth	Authenticated
Description	Mark notification(s) as read

Request Body:

```
{
  "notificationId": "string (mark one as read)",
  "markAllRead": "boolean (mark all as read)"
}
```

POST /api/notifications

Detail	Value
Auth	ADMIN only
Description	Create a notification for a user

Request Body:

```
{
  "userId": "string (required)",
  "title": "string (required)",
  "message": "string (required)",
  "type": "string (optional - e.g. 'info', 'session_reminder', 'payment_reminder')"
}
```

Onboarding

POST /api/onboarding

Detail	Value
Auth	Authenticated (any role)
Description	Save onboarding data and create student/teacher profile

Request Body (Student - type: "student"):

```
{
  "type": "student",
  "timezone": "string (required)",
  "country": "string (required)",
  "language": "string (required)",
  "trackType": "string (required - 'speaking_only' or 'full_student')",
  "level": "string (required - e.g. 'A1', 'B1', 'C1')",
  "availableSlots": "JSON string (optional - time slots)",
  "preferredTopics": "string (optional)",
  "internationalPreference": "boolean (default true)"
}
```

Request Body (Teacher - type: "teacher"):

```
{
  "type": "teacher",
  "timezone": "string (required)",
  "country": "string (required)",
  "specializations": "string (optional)",
  "bio": "string (optional)"
}
```

Teacher Resources

POST /api/teacher-resources/generate

Detail	Value
Auth	TEACHER or ADMIN
Description	AI-generate weekly teaching resources based on student analysis data

Request Body:

```
{
  "teacherId": "string (optional - defaults to session user)",
  "weekOf": "ISO date string (optional - defaults to current week)"
}
```

Response:

```
{
  "success": true,
  "weekOf": "ISO date",
  "resources": [{ "id", "resourceType", "title", "content", "targetGrammarPoint" }],
  "studentSpotlights": [{ "studentName", "note", "recommendation" }],
  "modelUsed": "string"
}
```

Cron Jobs

POST /api/cron/reminders

Detail	Value
Auth	Bearer token (CRON_SECRET env var)
Description	Automated reminder system. Checks sessions in next 48h, payments due in 3 days, overdue payments (7+ days), homework due in 24h. Sends WhatsApp + Email notifications.

Response:

```
{
  "success": true,
  "timestamp": "ISO date",
  "results": {
    "sessionReminders": "number",
    "paymentReminders": "number",
    "overduePayments": "number",
    "homeworkReminders": "number"
  },
  "total": "number"
}
```

Webhooks

POST /api/webhooks/stripe

Detail	Value
Auth	Stripe signature verification
Description	Handles payment_intent.succeeded and payment_intent.payment_failed. Confirms transactions and sends receipts.

POST /api/webhooks/papara

Detail	Value
Auth	None (webhook)
Description	Handles Papara payment notifications. Matches by AHK reference code in description. Auto-confirms and sends WhatsApp receipt.

Webhook Payload:

```
{
  "amount": "number (required)",
  "description": "string (should contain 'AHK-XXXX' reference)",
  "senderName": "string (optional)",
  "transactionId": "string (optional)",
  "createdAt": "ISO date (optional)"
}
```

POST /api/webhooks/fireflies

Detail	Value
Auth	None (webhook)
Description	Receives transcripts from Fireflies.ai. Matches to existing sessions by title/date. Creates placeholder session if no match.

Webhook Payload:

```
{
  "transcript_id": "string (optional)",
  "title": "string (optional)",
  "transcript_text": "string (required)",
  "date": "ISO date (optional)",
  "duration": "number (optional)",
  "participants": "array (optional)"
}
```

AI Chat

POST /api/chat

Detail	Value
Auth	None
Description	AI chatbot with cascade: Ollama (local) -> Groq -> Anthropic. Supports multiple bot personalities.

Request Body:

```
{
  "messages": [
    { "role": "user|assistant", "content": "string" }
  ],
  "bot": "string (bot identifier from BOT_CONFIG)"
}
```

Response:

```
{
  "message": "string (AI response)",
  "provider": "ollama|groq|anthropic|fallback"
}
```

Session Scheduling Helpers

POST /api/sessions/suggest-students

Detail	Value
Auth	TEACHER or ADMIN
Description	AI-powered student group suggestions based on level, availability, chemistry, diversity, and scheduling

Request Body:

```
{
  "sessionId": "string (optional - fetch from existing session)",
  "date": "ISO date (required if no sessionId)",
  "level": "string (required if no sessionId - e.g. 'B1')",
  "maxStudents": "number (default 5)"
}
```

Response:


```
{
  "suggestions": [{
    "studentId": "string",
    "name": "string",
    "level": "string",
    "country": "string|null",
    "score": "number",
    "reasons": ["string"]
  }]
}
```

POST /api/sessions/notify

Detail	Value
Auth	TEACHER or ADMIN
Description	Send session reminder notifications (48h or 2h before). Sends WhatsApp + Email to all participants.

Request Body:

```
{
  "sessionId": "string (required)",
  "type": "48h | 2h (required)"
}
```

Response:

```
{
  "success": true,
  "notificationsSent": "number",
  "type": "48h|2h",
  "sessionId": "string"
}
```

POST /api/sessions/auto-schedule

Detail	Value
Auth	TEACHER or ADMIN
Description	Find overlapping availability time slots for a group of students

Request Body:

```
{
  "studentIds": "string[] (required)",
  "durationMinutes": "number (required, min 15)",
  "preferredDays": "number[] (optional - 0=Sun, 1=Mon, etc.)"
}
```

Response:

```
{
  "slots": [{ "dayOfWeek": "number", "startTime": "HH:MM", "endTime": "HH:MM" }],
  "students": [{ "id", "name", "timezone", "hasAvailability" }],
  "durationMinutes": "number",
  "totalStudents": "number",
  "matchedStudents": "number"
}
```

Notes for Frontend Developers

Authentication Pattern

All protected endpoints use NextAuth.js session cookies. Call these endpoints from the browser after sign-in - the session cookie is sent automatically.

Error Format

All errors follow this pattern:

```
{
  "error": "Human-readable error message"
}
```

HTTP status codes: 400 (bad request), 401 (unauthorized), 403 (forbidden), 404 (not found), 500 (server error).

Common Patterns

- **Lists** return JSON arrays directly (e.g., GET /api/students returns [...])
- **Created resources** return the object with status 201
- **Deletions** return { success: true }
- **Role-based filtering** happens server-side based on the session

AI Cascade

AI endpoints (/api/analysis/generate, /api/homework/generate, /api/blog/generate, /api/teacher-resources/generate) use the AI cascade:

1. Gemini Flash (free tier)

2. Gemini Flash (paid)
3. Claude Haiku (fallback)

The `model` field in responses tells you which model was used.